

SQL

Interrogazioni di tipo insiemistico

Prof. Ionel Eduard Stan
2025/2026

Riadattamento delle slides del Prof. Simone Melzi

CAP 4

BASI DI DATI

ATZENI CERI PARABOSCHI TORLONE

Operatori insiemistici

Permettono di costruire query concatenando due query SQL

- **UNION**
- **INTERSECT**
- **EXCEPT** (differenza)

Sintassi:

SelectSQL { < UNION / INTERSECT / EXCEPT > [ALL] *SelectSQL* }

i duplicati vengono eliminati (a meno che si usi **ALL**)

Unione

Un'interrogazione o sottointerrogazione può essere costituita da due o più interrogazioni connesse dall'operatore UNION

- SELECT ...
- UNION [ALL]
- SELECT ...

l'operatore **UNION** restituisce tutte le tuple distinte restituite da almeno una delle sottointerrogazioni a cui è applicata

l'operatore **UNION** elimina i duplicati dal risultato

l'operatore **UNION ALL** non elimina i duplicati

Unione

l'operatore **UNION** impone alcune restrizioni sulle interrogazioni su cui opera:

- le interrogazioni devono restituire lo **stesso numero di colonne**, e le colonne corrispondenti devono avere lo stesso dominio (non è richiesto che abbiano la stessa lunghezza) o domini compatibili
- la corrispondenza si basa sulla **posizione delle colonne**, indipendentemente dal loro nome
- se si usa una clausola di **ORDER BY** questa deve essere usata **una sola volta alla fine dell'interrogazione** e *non alla fine di ogni SELECT*

Unione

R	A	B
	1	a
	1	a
	2	a
	2	b
	2	c
	3	b

S	C	B
	1	a
	1	b
	2	a
	2	c
	3	c
	4	d

le interrogazioni devono restituire lo **stesso numero di colonne**, e le colonne corrispondenti devono avere lo stesso dominio (*non è richiesto che abbiano la stessa lunghezza*) o domini compatibili

SELECT A	
FROM R	1
UNION	2
SELECT C	3
FROM S	4

SELECT A	A
FROM R	1
UNION	2
SELECT C AS A	3
FROM S	4

Unione

R

A	B
1	a
1	a
2	a
2	b
2	c
3	b

S

C	B
1	a
1	b
2	a
2	c
3	c
4	d

```
SELECT A,B
FROM R
UNION
SELECT B,C AS A
FROM S
```

Non corretta!

```
SELECT B
FROM R
UNION ALL
SELECT B
FROM S
```

B
a
a
a
a
b
c
b
a
c
c
d

la corrispondenza si basa sulla
posizione delle colonne,
indipendentemente dal loro nome

UNION

Cliente(<u>CodCli</u> , Indirizzo, P_IVA); Ordine(<u>CodOrd</u> , CodCli, Data, Importo) Dettaglio(<u>CodOrd</u> , <u>CodProd</u> , Qta) Prodotto(<u>CodProd</u> , Nome, Prezzo)

Query: Estrarre i codici degli ordini i cui importi superano 500 euro oppure in cui qualche prodotto è presente con quantità superiore a 1000

```
SELECT CodOrd  
FROM Ordine  
WHERE Importo > 500  
UNION
```

```
SELECT CodOrd  
FROM Dettaglio  
WHERE Qta > 1000
```

```
SELECT DISTINCT CodOrd  
FROM Ordine JOIN Dettaglio ON Ordine.CodOrd = Dettaglio.CodOrd  
WHERE Importo > 500 OR Qta > 1000
```

Notazione posizionale

paternita

padre	figlio
sergio	franco
luigi	olga
luigi	filippo
franco	andrea
franco	aldo

maternita

madre	figlio
luisa	maria
luisa	luigi
anna	olga
anna	filippo
maria	andrea
maria	aldo

```
SELECT padre
FROM paternita
UNION
SELECT madre
FROM maternita
```

padre
anna
franco
luigi
luisa
maria
sergio

Se gli attributi hanno un nome diverso il risultato normalmente non usa alcun nome o usa il nome del primo operando

Notazione posizionale, 2

Istanza:

padre	figlio
sergio	franco
luigi	olga
luigi	filippo
franco	andrea
franco	aldo

madre	figlio
luisa	maria
luisa	luigi
anna	olga
anna	filippo
maria	andrea
maria	aldo

Opzione1:

SELECT padre, figlio
FROM paternita
UNION
SELECT figlio, madre
FROM maternita

padre	figlio
aldo	maria
andrea	maria
filippo	anna
franco	aldo
franco	andrea
luigi	filippo
luigi	luisa
luigi	olga
maria	luisa
olga	anna
sergio	franco

Opzione2:

SELECT padre, figlio
FROM paternita
UNION
SELECT madre, figlio
FROM maternita

padre	figlio
anna	filippo
anna	olga
franco	aldo
franco	andrea
luigi	filippo
luigi	olga
luisa	luigi
luisa	maria
maria	aldo
maria	andrea
sergio	franco

Notazione posizionale, 3

- Anche con le ridenominazioni non cambia:

```
SELECT padre AS genitore, figlio
FROM paternita
UNION
SELECT figlio, madre AS genitore
FROM maternita
```

- Forma corretta:

```
SELECT padre AS genitore, figlio
FROM paternita
UNION
SELECT madre AS genitore, figlio
FROM maternita
```

genitore	figlio
aldo	maria
andrea	maria
filippo	anna
franco	aldo
franco	andrea
luigi	filippo
luigi	luisa
luigi	olga
maria	luisa
olga	anna
sergio	franco

genitore	figlio
anna	filippo
anna	olga
franco	aldo
franco	andrea
luigi	filippo
luigi	olga
luisa	luigi
luisa	maria
maria	aldo
maria	andrea
sergio	franco

Uso della parola chiave ALL

Query: Estrarre i padri di persone con nome “Giorgio” o “Giovanni”, presentando due volte i padri che hanno due figli (uno per ciascuno dei nomi)

```
SELECT Padre
FROM Paternita
WHERE Figlio = 'Giorgio'
      UNION ALL
SELECT Padre
FROM Paternita
WHERE Figlio = 'Giovanni'
```

Intersezione e differenza

- L'operatore **INTERSECT** esegue l'intersezione
- Gli operatori **EXCEPT** e **MINUS** eseguono la differenza

Per questi operatori valgono le stesse condizioni di applicabilità viste per l'operatore **UNION**

Le interrogazioni devono restituire lo **stesso numero di colonne**, e le colonne corrispondenti devono avere lo stesso dominio (non è richiesto che abbiano la stessa lunghezza) o domini compatibili

La corrispondenza si basa sulla **posizione delle colonne**, indipendentemente dal loro nome

Intersezione e differenza

R

A	B
1	a
1	a
2	a
2	b
2	c
3	b

S

C	B
1	a
1	b
2	a
2	c
3	c
4	d

```
SELECT B
FROM R
INTERSECT
SELECT B
FROM S
```

B
a
b
c

```
SELECT B
FROM S
EXCEPT
SELECT B
FROM R
```

B
d

```
SELECT B
FROM R
INTERSECT ALL
SELECT B
FROM S
```

B
a
a
b
c

```
SELECT B
FROM R
EXCEPT ALL
SELECT B
FROM S
```

B
a
b

Differenza

Impiegato

ID impiegato	Nome	Cognome	Dipart	Ufficio	Stipendio	premioprod	Mansione	Città	idcapo
--------------	------	---------	--------	---------	-----------	------------	----------	-------	--------

dip

Nome	Indirizzo	Città
------	-----------	-------

Query: Estrarre il nome degli impiegati esclusi quelli che hanno il nome uguale al cognome di un altro impiegato

```
SELECT Nome
FROM Impiegato
EXCEPT
SELECT Cognome AS Nome
FROM Impiegato
```

- Si può fare con join
- vedremo che si può esprimere con **SELECT** nidificate

Intersezione

impiegato

ID impiegato	Nome	Cognome	Dipart	Ufficio	Stipendio	premioprod	Mansione	Città	idcapo
--------------	------	---------	--------	---------	-----------	------------	----------	-------	--------

dip

Nome	Indirizzo	Città
------	-----------	-------

Query: Estrarre gli impiegati che hanno il nome uguale al cognome di un altro impiegato

```
SELECT Nome
FROM Impiegato
      INTERSECT
SELECT Cognome AS Nome
FROM Impiegato
```

equivale a:

```
SELECT DISTINCT I.Nome
FROM Impiegato AS I, Impiegato AS J
WHERE I.Nome = J.Cognome
```

Intersezione

```
Cliente(CODCLI, INDIRIZZO, P_IVA);  
Ordine(CODORD, CODCLI, DATA, IMPORTO)  
Dettaglio(CODORD, CODPROD, QTA)  
Prodotto(CODPROD, NOME, PREZZO)
```

Query: Estrarre i codici degli ordini i cui importi superano 500 euro e in cui qualche prodotto è presente con quantità superiore a 1000

```
SELECT CodOrd  
FROM Ordine  
WHERE Importo > 500  
INTERSECT  
SELECT CodOrd  
FROM Dettaglio  
WHERE Qta > 1000
```


ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

Query: Le città con un aeroporto di cui non è noto il numero di piste

```
SELECT Città  
FROM AEROPORTO  
WHERE NumPiste IS NULL
```

ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

Query: Le nazioni da cui parte o arriva il volo con codice AZ274

```
SELECT A1.Nazione, A2.Nazione
```

```
FROM AEROPORTO AS A1 JOIN VOLO ON A1.Città = CittàArr
```

```
JOIN AEROPORTO AS A2 ON CittàPart = A2.Città
```

```
WHERE IdVolo = 'AZ274'
```

ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

Query: I tipi di aereo e il corrispondente numero di passeggeri per i tipi di aereo usati nei voli che partono da Torino. Se la descrizione dell'aereo non è disponibile, visualizzare solamente il tipo;

```
SELECT VOLO.TipoAereo, AEREO.NumPasseggeri
FROM VOLO LEFT JOIN AEREO
      ON VOLO.TipoAereo=AEREO.TipoAereo
WHERE CittàPart= 'Torino'
```

ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

Query: Le città da cui partono voli internazionali

SELECT VOLO.CittàPart

FROM AEROPORTO **AS** A1 **JOIN** VOLO **ON** CittàPart=A1.Città

JOIN AEROPORTO **AS** A2 **ON** CittàArr=A2.Città

WHERE A1.Nazione <> A2.Nazione

ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

Query: Le città da cui partono voli diretti a Bologna, ordinate alfabeticamente;

```
SELECT CittàPart
FROM VOLO
WHERE CittàArr= 'Bologna'
ORDER BY CittàPart
```

ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

Query: Il numero di voli internazionali che partono ogni settimana da ogni città italiana

```
SELECT COUNT(*), CittàPart
FROM AEROPORTO AS A1 JOIN VOLO ON A1.Città = CittàPart
      JOIN AEROPORTO AS A2 ON CittàArr = A2.Città
WHERE A1.Nazione = 'Italia' AND A2.Nazione <> 'Italia'
GROUP BY CittàPart
```

ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

Query: Il numero di voli internazionali che partono ogni settimana da città italiane (facendo comparire nel risultato gli eventuali aeroporti senza voli internazionali);

```
SELECT CittàPart COUNT (*)  
FROM AEROPORTO AS A1 LEFT JOIN VOLO ON A1.Città = CittàPart  
JOIN AEROPORTO AS A2 ON CittàArr = A2.Città  
WHERE A1.Nazione = 'Italia' AND (A2.Nazione <> 'Italia' OR A2.Nazione IS  
NULL)  
GROUP BY CittàPart
```


ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

Query: Il numero di voli internazionali che partono il giovedì da Napoli;

```
SELECT COUNT(*)  
FROM VOLO JOIN AEROPORTO ON CittàArr = Città  
WHERE CittàPart = 'Napoli' AND Nazione <> 'Italia' AND  
      GiornoSett = 'Giovedì'
```

ESERCIZIO

Dato il seguente schema:

AEROPORTO(Città, Nazione, NumPiste)

VOLO(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, TipoAereo)

AEREO(TipoAereo, NumPasseggeri, QtaMerci)

scrivere le interrogazioni SQL che permettono di determinare:

Le città francesi da cui partono più di venti voli alla settimana diretti in Italia;

```
SELECT CittàPart
FROM AEROPORTO AS A1 JOIN VOLO ON A1.Città = CittàPart
      JOIN AEROPORTO AS A2 ON CittàArr = A2.Città
WHERE A1.Nazione = 'Francia' AND A2.Nazione = 'Italia'
GROUP BY CittàPart
HAVING COUNT(*) >20
```